

Verilog HDL - Part B

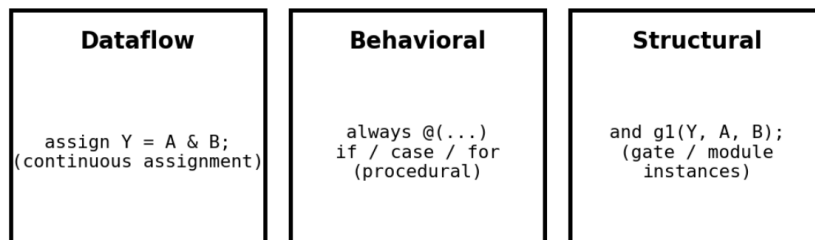
Questions 238 – 276 | Dataflow, Structural and Behavioural modelling | ECE S3

Question	Topic	Name	Roll No	Page
Q 238 B	Three types of modeling in Verilog	Renjitha Babu	55	2
Q 239 B	Half adder – dataflow & structural	Renjitha Babu	55	3
Q 240 B	Full adder – structural (gates)	Rihan S Mohammed	56	5
Q 241 B	Full adder – two half adders (structural)	Rihan S Mohammed	56	6
Q 242 B	Full subtractor – structural	Robin Kirian Abraham	57	7
Q 243 B	Full adder – NAND gates only	Robin Kirian Abraham	57	9
Q 244 B	Full subtractor – NAND gates only	Sethuparvathy K J	58	11
Q 245 B	SR flip-flop – structural	Sethuparvathy K J	58	13
Q 246 B	Full adder from two half adders – module instantiation	Shaheem Ahamed K	59	15
Q 247 B	JK flip-flop – structural	Shaheem Ahamed K	59	17
Q 248 B	K-map simplification (3 variables, don't-cares)	Sharika V	60	18
Q 249 B	K-map simplification (4 variables)	Sharika V	60	20
Q 250 B	4:1 multiplexer – structural	Sidharth P	62	23
Q 251 B	1:4 demultiplexer – structural	Sidharth P	62	25
Q 252 B	8:1 multiplexer – structural	Sidharth S	63	27
Q 253 B	1:8 demultiplexer – structural	Sidharth S	63	29
Q 254 B	3:8 decoder – structural	Sivapriya K G	64	31
Q 255 B	4-bit adder – module instantiation	Sivapriya K G	64	33
Q 256 B	4-bit adder / subtractor – behavioural	Sreya Manoj	66	34
Q 257 B	8:1 multiplexer – behavioural (case)	Sreya Manoj	66	35
Q 258 B	JK flip-flop – behavioural (case)	Thejus Mary Biju	67	36
Q 259 B	JK flip-flop – behavioural (if-else)	Thejus Mary Biju	67	38
Q 260 B	4-bit up counter – behavioural	Thomas	68	39
Q 261 B	4-bit down counter – behavioural	Thomas	68	41
Q 262 B	Decade counter – behavioural	Upanisha	69	43
Q 272 B	Half subtractor – dataflow & structural	Upanisha	69	46
Q 275 B	Full adder – structural (gates)	Vasudev S	70	48
Q 276 B	Full adder – two half adders (structural)	Vasudev S	70	49

Q 238 B**Explain the three main types of modeling in Verilog, with examples.**

Verilog describes hardware at different levels of abstraction. The three main modeling styles are dataflow, behavioral and structural.

Verilog modeling styles (abstraction)

**1. Dataflow modeling**

The circuit is described by how data flows between signals, using continuous assignments (**assign**) and operators. The output updates whenever any input changes.

```
module and_df (input a, b, output y);
    assign y = a & b;
endmodule
```

2. Behavioral modeling

The circuit is described by its behavior (algorithm) using procedural blocks (**always / initial**) with constructs such as if-else, case and loops. It is the highest level of abstraction and is used for both combinational and sequential logic.

```
module mux2 (input a, b, s, output reg y);
    always @(a or b or s)
        if (s) y = b;
        else y = a;
endmodule
```

3. Structural (gate-level) modeling

The circuit is described as an interconnection of built-in gate primitives (and, or, not, xor, etc.) or of other module instances, exactly like a schematic.

```
module and_st (input a, b, output y);
    and g1 (y, a, b);
endmodule
```

Style	Keyword / construct	Abstraction
Dataflow	assign, operators	Medium
Behavioral	always, initial, if / case	Highest
Structural	Gate primitives, module instances	Lowest

Q 239 B

Write Verilog code for a half adder using both Dataflow modeling and Structural modeling.

A half adder adds two 1-bit inputs A and B and produces Sum and Carry.

Truth table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean expressions: Sum = $A \oplus B$, Carry = $A \cdot B$

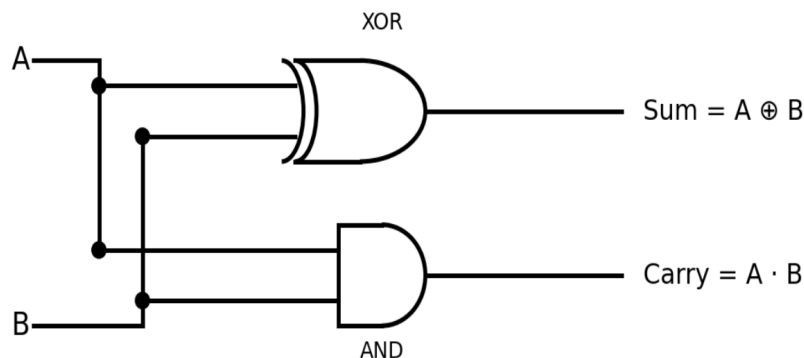


Fig. 1: Half adder (XOR + AND)

(a) Dataflow modeling

```
module half_adder_df (input a, b, output sum, carry);
    assign sum = a ^ b;
    assign carry = a & b;
endmodule
```

(b) Structural modeling

```
module half_adder_st (input a, b, output sum, carry);
    xor (sum, a, b);
    and (carry, a, b);
endmodule
```

(c) Structural modeling using only NAND gates

A half adder can be built from 5 NAND gates: $N1 = (A \cdot B)'$, $N2 = (A \cdot N1)'$, $N3 = (B \cdot N1)'$, Sum = $(N2 \cdot N3)' = A \oplus B$, Carry = $(N1 \cdot N1)' = A \cdot B$.

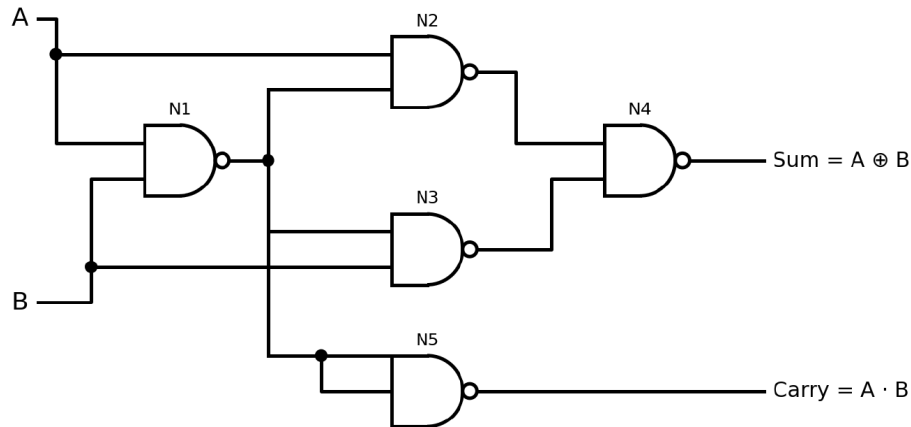


Fig. 2: Half adder using five NAND gates

```

module half_adder_nand (input a, b, output sum, carry);
    wire n1, n2, n3;          // internal wires
    nand (n1, a, b);
    nand (n2, a, n1);
    nand (n3, b, n1);
    nand (sum, n2, n3);
    nand (carry, n1, n1);
endmodule

```

(d) Dataflow modeling using only NAND logic

The same 5-NAND circuit as above, written with continuous assignments. Each NAND gate is expressed as $\sim(x \& y)$.

```

module half_adder_nand_df (input a, b, output sum, carry);
    wire n1, n2, n3;
    assign n1    = ~(a & b);
    assign n2    = ~(a & n1);
    assign n3    = ~(b & n1);
    assign sum   = ~(n2 & n3); // = a ^ b
    assign carry = ~(n1 & n1); // = a & b
endmodule

```

Q 240 B

Write Verilog code for a Full adder using structural modelling. Draw the gate-level circuit also.

A full adder adds three 1-bit inputs A, B and Cin and gives two outputs, Sum and Cout. In structural modelling it is built by instantiating the gate primitives **xor**, **and**, **or** and joining them with wires.

Boolean equations: $\text{Sum} = A \oplus B \oplus \text{Cin}$ and $\text{Cout} = A \cdot B + \text{Cin} \cdot (A \oplus B)$.

Verilog program - Structural modelling

```
module full_adder(input A, B, Cin, output Sum, Cout);
    wire P, G, T;          // internal wires

    xor (P, A, B);          // P = A xor B
    xor (Sum, P, Cin);      // Sum = P xor Cin
    and (G, A, B);          // G = A . B
    and (T, P, Cin);        // T = P . Cin
    or  (Cout, G, T);       // Cout = G + T
endmodule
```

Working: The first XOR gate gives $P = A \oplus B$. The second XOR gate combines P with Cin to give Sum. The two AND gates generate $A \cdot B$ and $P \cdot \text{Cin}$, and the OR gate adds them to produce the carry-out.

Truth table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

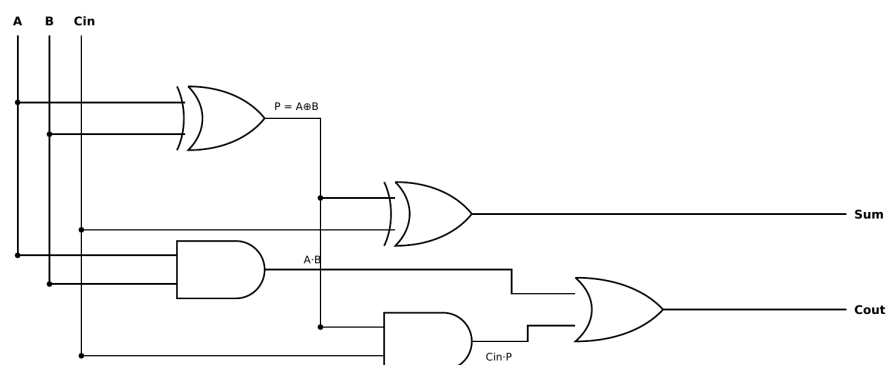
Gate-level circuit

Fig. 1: Full adder (2 XOR, 2 AND, 1 OR)

Operation: Sum is 1 when an odd number of inputs are 1. Cout is 1 when at least two inputs are 1.

Q 241 B

Write Verilog code for a Full adder using structural modelling (two half adders and one OR gate). Draw the gate-level circuit also.

A full adder can also be built structurally from **two half adders and one OR gate**. A half adder (xor + and) is first written as a separate module, and it is instantiated twice inside the full adder module (hierarchical structural modelling). HA1 adds A and B; HA2 adds the partial sum S1 to Cin; the two carries C1 and C2 are ORed to get Cout.

Verilog program - Structural modelling

```
module half_adder(input a, b, output s, c);
    xor (s, a, b);          // sum = a xor b
    and (c, a, b);          // carry = a . b
endmodule

module full_adder_ha(input A, B, Cin, output Sum, Cout);
    wire S1, C1, C2;        // internal wires

    half_adder HA1 (A, B, S1, C1);
    half_adder HA2 (S1, Cin, Sum, C2);
    or (Cout, C1, C2);      // Cout = C1 + C2
endmodule
```

Working: HA1 produces $S1 = A \oplus B$ and $C1 = A \cdot B$. HA2 produces $Sum = S1 \oplus Cin$ and $C2 = S1 \cdot Cin$. Finally $Cout = C1 + C2$, which equals $A \cdot B + Cin \cdot (A \oplus B)$, the same as the full adder equation.

Operation: Each dashed box is one half adder (an XOR gate and an AND gate). Two half adders plus an OR gate give the complete full adder.

Truth table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

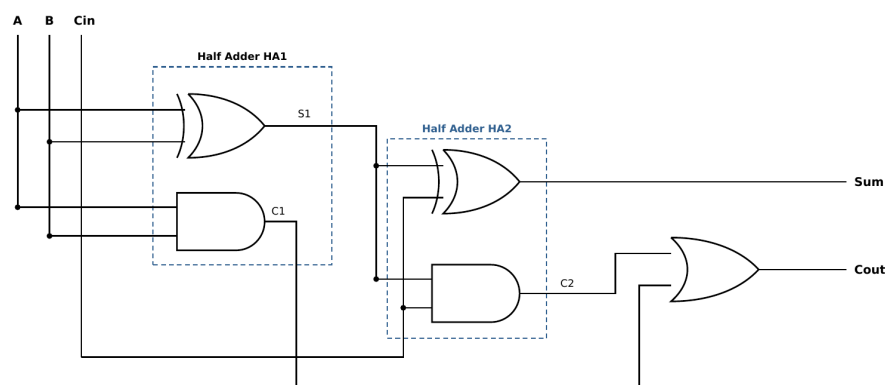
Gate-level circuit

Fig. 1: Full adder from two half adders and an OR gate

Q 242 B

Write Verilog code for a Full subtractor using Structural modelling. Draw the gate-level circuit also.

Theory

A full subtractor subtracts three one-bit inputs: A (minuend), B (subtrahend) and Bin (borrow in). It produces the Difference (D) and the Borrow out (Bout).

$$D = A \oplus B \oplus \text{Bin}$$

$$\text{Bout} = A'B + (A \oplus B) \cdot \text{Bin} = A'B + (A \oplus B)' \cdot \text{Bin}$$

Truth table

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

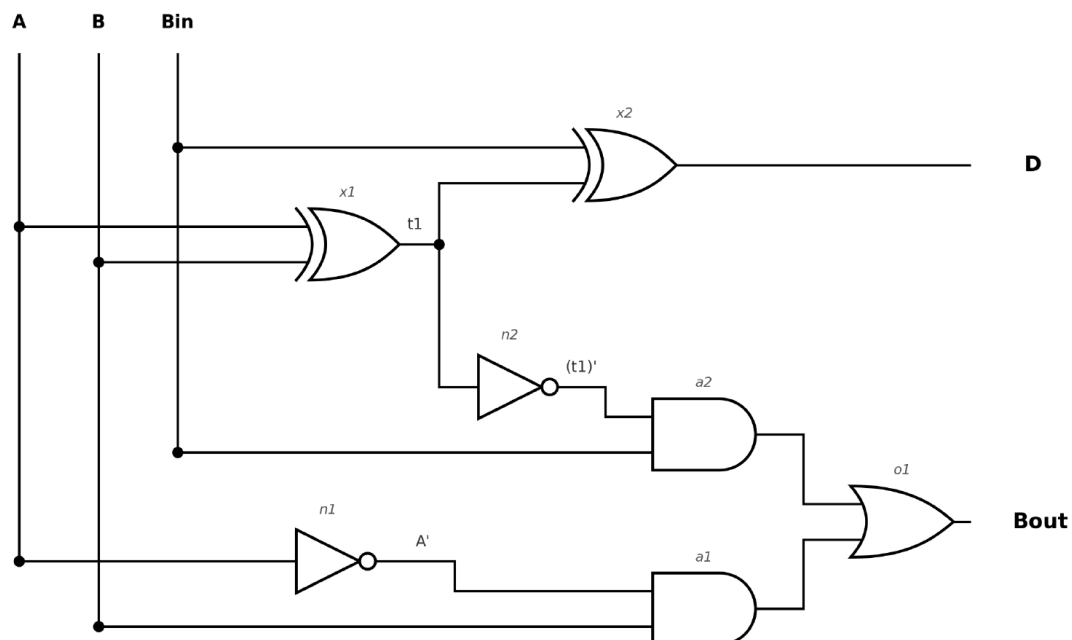
Gate-level circuit

Fig. 1: Full subtractor (2 XOR, 2 NOT, 2 AND, 1 OR)

Verilog code (Structural modelling)

```
module full_subtractor(input A, B, Bin, output D, Bout);
    wire t1, nA, nt1, t2, t3;

    xor x1(t1, A, B);           // t1 = A ^ B
    xor x2(D, t1, Bin);         // D = t1 ^ Bin
    not n1(nA, A);              // nA = A'
    and a1(t2, nA, B);          // t2 = A'B
    not n2(nt1, t1);            // nt1 = (A ^ B)'
    and a2(t3, nt1, Bin);        // t3 = (A ^ B)' Bin
    or o1(Bout, t2, t3);        // Bout = t2 + t3
endmodule
```

Q 243 B

Write Verilog code for a Full adder using NAND gates only. Use Structural modelling. Draw the gate-level circuit also.

Theory

A full adder adds three one-bit inputs A, B and Cin, giving Sum and Cout. NAND is a universal gate, so the whole adder can be built from 9 two-input NAND gates.

$$\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$$

$$\text{Cout} = AB + \text{Cin}(A \text{ XOR } B)$$

Truth table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

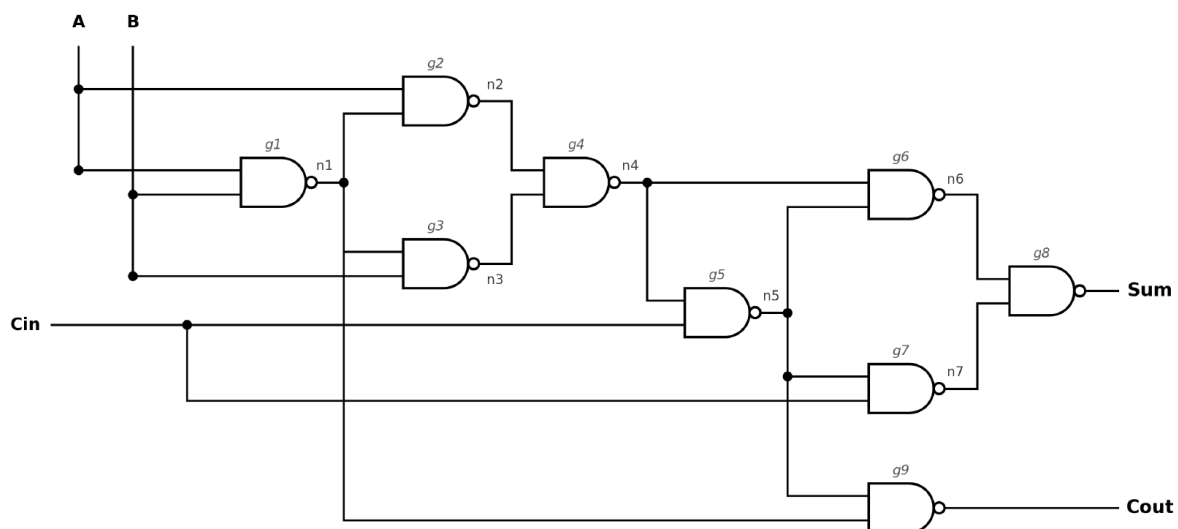
Gate-level circuit

Fig. 1: Full adder using only NAND gates (g1 to g9)

How the NAND network works: g1 to g4 form A XOR B (n4). g5 to g8 repeat the same XOR structure with n4 and Cin to give Sum. Cout = $(n5 \cdot n1)' = AB + \text{Cin}(A \text{ XOR } B)$.

Verilog code (Structural modelling)

```
module full_adder_nand(input A, B, Cin, output Sum, Cout);
    wire n1, n2, n3, n4, n5, n6, n7;

    nand g1(n1, A, B);
    nand g2(n2, A, n1);
    nand g3(n3, B, n1);
    nand g4(n4, n2, n3);          // n4 = A ^ B
    nand g5(n5, n4, Cin);
    nand g6(n6, n4, n5);
    nand g7(n7, Cin, n5);
    nand g8(Sum, n6, n7);         // Sum = A ^ B ^ Cin
    nand g9(Cout, n5, n1);        // Cout = AB + Cin(A ^ B)
endmodule
```

Q 244 B

Write Verilog code for a Full Subtractor using NAND gates only. Use Structural modelling. Draw the gate-level circuit also.

1. Truth table

A	B	Bin	Difference (D)	Borrow (Bout)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2. Boolean expressions

$$D = A \oplus B \oplus \text{Bin}$$

$$\text{Bout} = A'B + (A \oplus B)' \cdot \text{Bin}$$

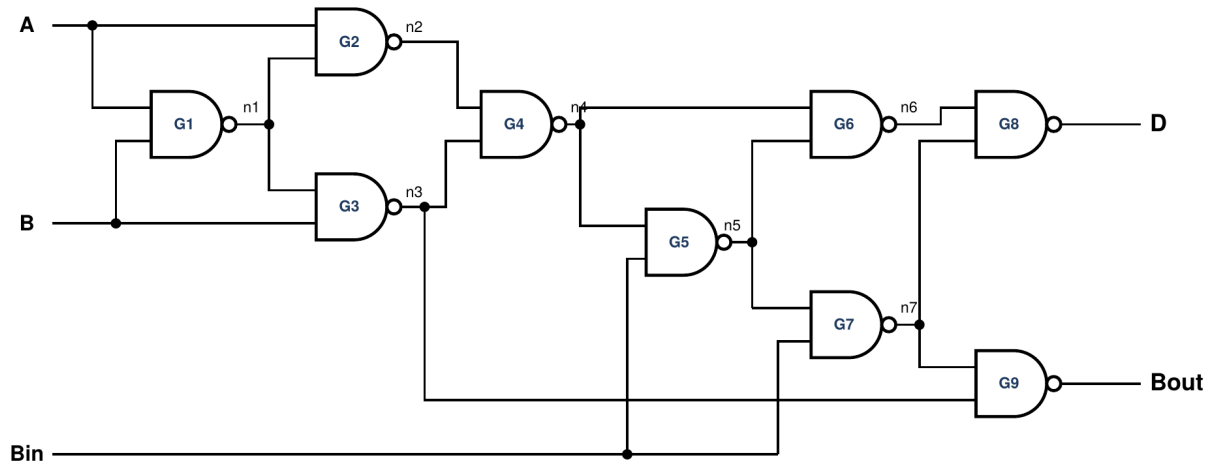
3. Gate-level circuit (NAND only)

Fig. 1: Full subtractor using nine 2-input NAND gates

4. NAND-only realisation (9 NAND gates)

Using NAND as the universal gate, the XOR is built from 4 NAND gates and the whole circuit shares intermediate terms:

Gate	Output	Function	Meaning
G1	n1	$(A \cdot B)'$	common term
G2	n2	$(A \cdot n1)'$	-
G3	n3	$(B \cdot n1)'$	$= (A'B)'$
G4	n4	$(n2 \cdot n3)'$	$= A \text{ XOR } B$
G5	n5	$(n4 \cdot \text{Bin})'$	common term
G6	n6	$(n4 \cdot n5)'$	-
G7	n7	$(\text{Bin} \cdot n5)'$	$= (\text{Bin} \cdot n4)'$
G8	D	$(n6 \cdot n7)'$	$= A \text{ XOR } B \text{ XOR } \text{Bin}$
G9	Bout	$(n3 \cdot n7)'$	$= A'B + (A \text{ XOR } B)' \text{Bin}$

5. Verilog code - Structural modelling

```

module full_subtractor_nand (
    input A, B, Bin,
    output D, Bout
);
    wire n1, n2, n3, n4, n5, n6, n7;    // internal nets

    nand G1 (n1, A, B);
    nand G2 (n2, A, n1);
    nand G3 (n3, B, n1);
    nand G4 (n4, n2, n3);    // n4 = A xor B
    nand G5 (n5, n4, Bin);
    nand G6 (n6, n4, n5);
    nand G7 (n7, Bin, n5);
    nand G8 (D, n6, n7);    // Difference
    nand G9 (Bout, n3, n7); // Borrow
endmodule

```

6. Test bench

```

module tb_full_subtractor_nand;
    reg A, B, Bin;
    wire D, Bout;

    full_subtractor_nand uut (.A(A), .B(B), .Bin(Bin), .D(D), .Bout(Bout));

    initial begin
        $monitor("%0t A=%b B=%b Bin=%b | D=%b Bout=%b",
            $time, A, B, Bin, D, Bout);
        {A, B, Bin} = 3'b000; #10;
        {A, B, Bin} = 3'b001; #10;
        {A, B, Bin} = 3'b010; #10;
        {A, B, Bin} = 3'b011; #10;
        {A, B, Bin} = 3'b100; #10;
        {A, B, Bin} = 3'b101; #10;
        {A, B, Bin} = 3'b110; #10;
        {A, B, Bin} = 3'b111; #10;
        $finish;
    end
endmodule

```

7. Result

The simulated outputs match the truth table: $D = A \text{ XOR } B \text{ XOR } \text{Bin}$ and $\text{Bout} = A'B + (A \text{ XOR } B)' \text{Bin}$. The full subtractor was implemented using only NAND gates with structural modelling.

Q 245 B

Draw the gate-level circuit of SR Flip-flop and implement the same in Verilog HDL using Structural modelling.

1. Working principle

A clocked SR flip-flop is a cross-coupled NAND latch (G3, G4) preceded by a pair of steering NAND gates (G1, G2) that are enabled by the clock. When $CLK = 0$ the steering gates output 1 and the latch holds its state. When $CLK = 1$ the inputs S (Set) and R (Reset) control the state.

2. Characteristic (function) table

CLK	S	R	Q(next)	Q'(next)	Operation
0	X	X	Q	Q'	No change (hold)
1	0	0	Q	Q'	No change (hold)
1	0	1	0	1	Reset
1	1	0	1	0	Set
1	1	1	1*	1*	Invalid / forbidden

* Both outputs go to 1 and the next state is unpredictable when CLK returns to 0, so $S = R = 1$ must be avoided.

Characteristic equation: $Q(\text{next}) = S + R'Q$, with constraint $S \cdot R = 0$

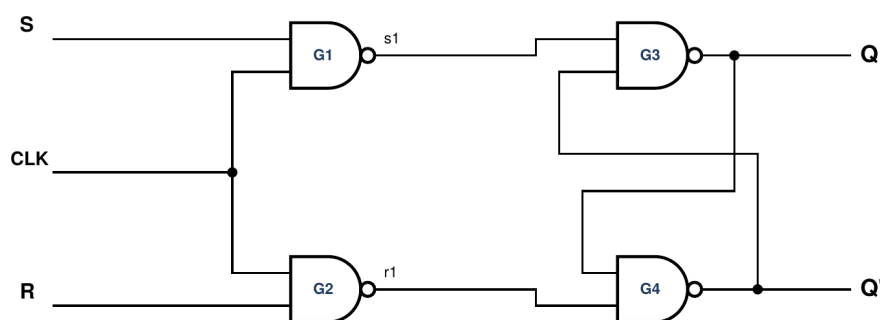
3. Gate-level circuit

Fig. 1: Clocked SR flip-flop using four 2-input NAND gates

4. Verilog code - Structural modelling

```

module sr_flipflop (
    input S, R, CLK,
    output Q, Qb
);
    wire s1, r1;                // outputs of steering gates

    nand G1 (s1, S, CLK);        // steering gate for Set
    nand G2 (r1, R, CLK);        // steering gate for Reset
    nand G3 (Q, s1, Qb);         // cross-coupled latch
    nand G4 (Qb, r1, Q);
endmodule

```

5. Test bench

```
module tb_sr_flipflop;
    reg S, R, CLK;
    wire Q, Qb;

    sr_flipflop uut (.S(S), .R(R), .CLK(CLK), .Q(Q), .Qb(Qb));

    always #5 CLK = ~CLK;      // clock, period = 10 ns

    // inputs change while CLK = 0
    initial begin
        $monitor("%0t CLK=%b S=%b R=%b | Q=%b Qb=%b",
            $time, CLK, S, R, Q, Qb);
        CLK = 0; S = 0; R = 1; #12;    // Reset -> Q = 0
        S = 0; R = 0; #10;             // Hold  -> Q = 0
        S = 1; R = 0; #10;             // Set   -> Q = 1
        S = 0; R = 0; #10;             // Hold  -> Q = 1
        S = 0; R = 1; #10;             // Reset -> Q = 0
        S = 1; R = 0; #10;             // Set   -> Q = 1
        $finish;
    end
endmodule
```

6. Result

The SR flip-flop was designed at gate level with four NAND gates and implemented in Verilog using structural modelling. Its behaviour (hold, reset, set, and the invalid state at $S = R = 1$) agrees with the function table.

Q 246 B

Implement a full adder using TWO half adders and necessary gates. Write the Verilog HDL model for this full adder circuit using module instantiation.

A full adder adds three 1-bit inputs (A, B and carry-in Cin) and produces Sum and Carry-out (Cout). It can be built from two half adders and one OR gate:

- **Half adder 1** adds A and B: $s1 = A \oplus B$, $c1 = A \cdot B$
- **Half adder 2** adds s1 and Cin: $\text{Sum} = s1 \oplus \text{Cin}$, $c2 = s1 \cdot \text{Cin}$
- **OR gate** combines the carries: $\text{Cout} = c1 + c2$

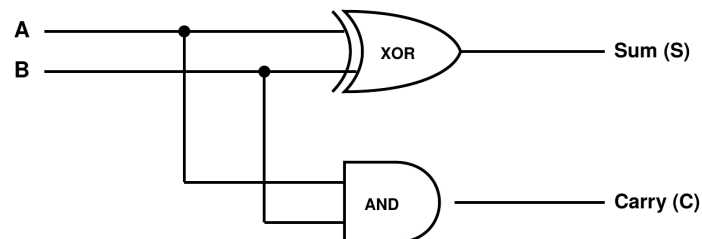
Half adder (gate-level circuit)

Fig. 1: Half adder

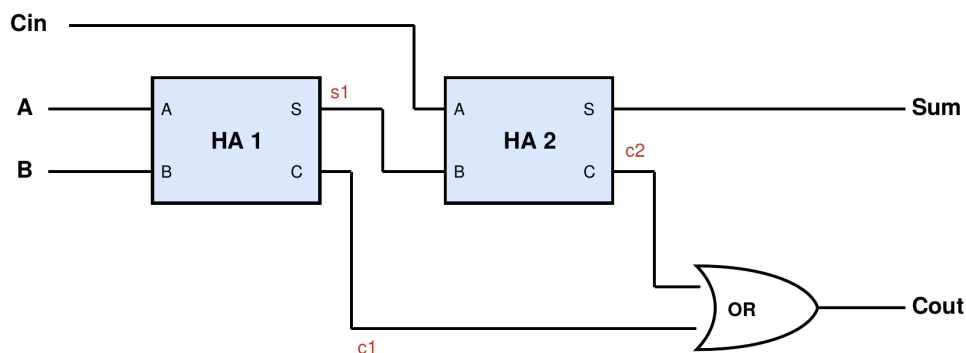
Full adder using two half adders and an OR gate

Fig. 2: Full adder built from HA 1, HA 2 and an OR gate

Truth table of the full adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Verilog HDL code - module instantiation

```
// Half adder module (gate-level)
module half_adder(input a, b, output s, c);
    xor x1(s, a, b);
    and a1(c, a, b);
endmodule

// Full adder using two half adders and one OR gate
module full_adder(input a, b, cin, output sum, cout);
    wire s1, c1, c2;                                // internal nets

    half_adder ha1(.a(a), .b(b), .s(s1), .c(c1));    // instance 1
    half_adder ha2(.a(s1), .b(cin), .s(sum), .c(c2)); // instance 2
    or o1 (cout, c1, c2);                            // OR gate
endmodule
```

The **half_adder** module is instantiated twice inside **full_adder** (module instantiation) and the internal wires s1, c1, c2 connect the blocks together.

Q 247 B**Draw the gate-level circuit of JK Flip-flop and implement the same in Verilog HDL using Structural modelling.**

A JK flip-flop is built from four NAND gates: two 3-input NAND gates (G1, G2) that steer the J and K inputs with the clock, and two cross-coupled 2-input NAND gates (G3, G4) forming the latch. The outputs Q and Q' are fed back to the input gates.

- $S = (J \cdot \text{CLK} \cdot Q')'$ (G1)
- $R = (K \cdot \text{CLK} \cdot Q)'$ (G2)
- $Q = (S \cdot Q')'$ (G3), $Q' = (R \cdot Q)'$ (G4)

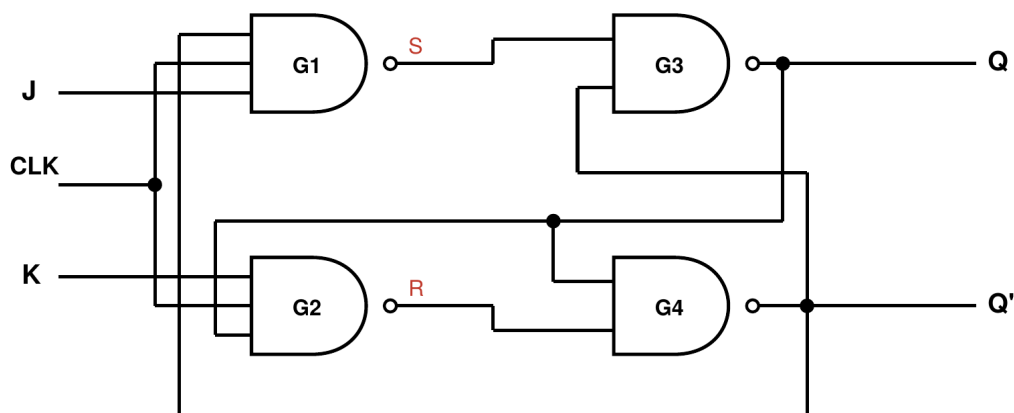
Gate-level circuit

Fig. 1: JK flip-flop using four NAND gates (crossing wires without a dot are not connected)

J	K	Q(next)	Operation
0	0	Q	No change
0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle

Verilog HDL code (structural modelling)

```

module jk_ff(input j, k, clk, output q, qbar);
    wire s, r;                // outputs of input-steering gates

    nand g1(s, j, clk, qbar);  // 3-input NAND
    nand g2(r, k, clk, q);     // 3-input NAND
    nand g3(q, s, qbar);       // cross-coupled latch
    nand g4(qbar, r, q);
endmodule

```

Note: this is a level-triggered JK flip-flop, so with $J = K = 1$ it can toggle repeatedly while CLK is high (race-around). A master-slave or edge-triggered design (or a narrow clock pulse) avoids this. In simulation q and qbar start as x, so apply $J = 0, K = 1$ with a clock pulse first to reset it.

Q 248 B

Simplify the Boolean function $F(A, B, C) = \sum m(1, 3, 4, 7) + d(0, 5)$ and implement it using Verilog HDL.

Step 1 - Truth table

Minterm	A	B	C	F
m0	0	0	0	X
m1	0	0	1	1
m2	0	1	0	0
m3	0	1	1	1
m4	1	0	0	1
m5	1	0	1	X
m6	1	1	0	0
m7	1	1	1	1

Step 2 - Karnaugh map

$A \backslash BC$	00	01	11	10
0	⁰ X	¹ 1	³ 1	² 0
1	⁴ 1	⁵ X	⁷ 1	⁶ 0

B' (quad m_0, m_1, m_4, m_5)
 C (quad m_1, m_3, m_5, m_7)
 X = don't care

Fig. 1: K-map of F. The don't-cares m_0 and m_5 are treated as 1 so that the largest groups can be formed.

Both groups are quads (largest possible) and are essential: m_4 is covered only by the B' quad and m_3, m_7 only by the C quad.

$$F(A, B, C) = B' + C$$

Check: $m_1, m_3, m_7 \rightarrow C = 1$; $m_4 \rightarrow B' = 1$; m_2, m_6 ($B = 1, C = 0$) $\rightarrow F = 0$ as required. The don't-cares evaluate to $m_0 \rightarrow 1, m_5 \rightarrow 1$. Variable A does not appear in the simplified function.

Step 3 - Verilog implementation

```
// Gate-level (structural) model:  $F = B' + C$ 
module f248_gate (output F, input A, B, C); // A is unused in the simplified function
    wire Bn;
    not N1 (Bn, B);
    or O1 (F, Bn, C);
endmodule

// Dataflow model
module f248_df (output F, input A, B, C);
    assign F = ~B | C;
endmodule

// Test bench
module tb_f248;
    reg A, B, C; wire F;
    f248_gate DUT (.F(F), .A(A), .B(B), .C(C));
    initial begin
        $monitor("t=%0t ABC=%b%b%b F=%b", $time, A, B, C, F);
        {A,B,C} = 3'b000;
        repeat (7) #10 {A,B,C} = {A,B,C} + 1;
        #10 $finish;
    end
endmodule
```

Gate-level circuit

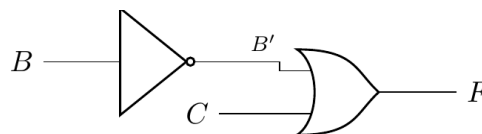


Fig. 2: $F = B' + C$ (one NOT gate and one OR gate)

Simulation timing diagram

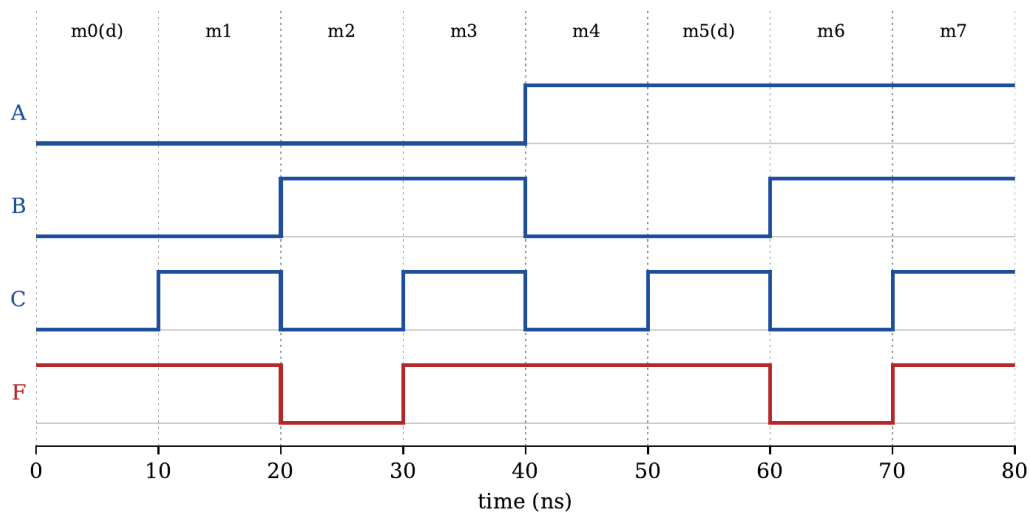


Fig. 3: ABC counts 000 to 111 every 10 ns; $F = 1$ for m0, m1, m3, m4, m5, m7 and $F = 0$ for m2, m6.

Q 249 B

Simplify the Boolean function $F(A, B, C, D) = \sum m(0, 3, 6, 7, 8, 10, 12, 15)$ and implement it using Verilog HDL.

Step 1 - Truth table

Minterm	A	B	C	D	F
m0	0	0	0	0	1
m1	0	0	0	1	0
m2	0	0	1	0	0
m3	0	0	1	1	1
m4	0	1	0	0	0
m5	0	1	0	1	0
m6	0	1	1	0	1
m7	0	1	1	1	1
m8	1	0	0	0	1
m9	1	0	0	1	0
m10	1	0	1	0	1
m11	1	0	1	1	0
m12	1	1	0	0	1
m13	1	1	0	1	0
m14	1	1	1	0	0
m15	1	1	1	1	1

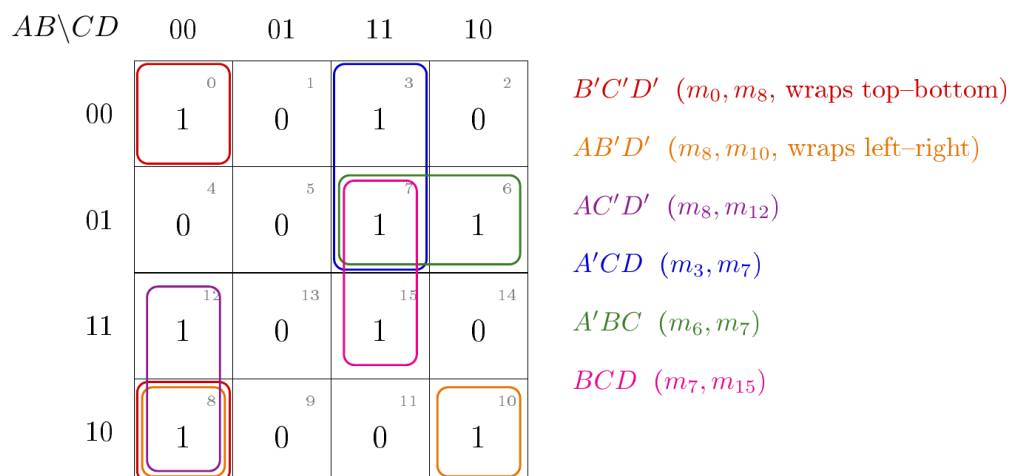
Step 2 - Karnaugh map

Fig. 1: K-map of $F(A, B, C, D)$ with all prime implicants (no quads or octets exist).

Step 3 - Prime implicants

Prime implicant	Minterms	Essential because it alone covers
B'C'D'	m0, m8	m0
AB'D'	m8, m10	m10
AC'D'	m8, m12	m12
A'CD	m3, m7	m3
A'BC	m6, m7	m6
BCD	m7, m15	m15

All six prime implicants are essential and together cover all eight minterms, so the minimal sum-of-products is unique:

$$F = B'C'D' + AB'D' + AC'D' + A'CD + A'BC + BCD$$

Step 4 - Verilog implementation

```
// Gate-level (structural) model: 6 AND gates + 1 OR gate
module f249_gate (output F, input A, B, C, D);
    wire An, Bn, Cn, Dn;           // complemented inputs
    wire t1, t2, t3, t4, t5, t6;   // product terms
    not N1 (An, A); not N2 (Bn, B); not N3 (Cn, C); not N4 (Dn, D);
    and A1 (t1, Bn, Cn, Dn);        // B'C'D'
    and A2 (t2, A, Bn, Dn);         // AB'D'
    and A3 (t3, A, Cn, Dn);         // AC'D'
    and A4 (t4, An, C, D);          // A'CD
    and A5 (t5, An, B, C);          // A'BC
    and A6 (t6, B, C, D);           // BCD
    or O1 (F, t1, t2, t3, t4, t5, t6);
endmodule

// Dataflow model
module f249_df (output F, input A, B, C, D);
    assign F = (~B & ~C & ~D) | (A & ~B & ~D) | (A & ~C & ~D)
               | (~A & C & D) | (~A & B & C) | (B & C & D);
endmodule

// Test bench: apply all 16 input combinations
module tb_f249;
    reg A, B, C, D; wire F;
    f249_gate DUT (.F(F), .A(A), .B(B), .C(C), .D(D));
    initial begin
        $monitor("t=%0t ABCD=%b%b%b%b F=%b", $time, A, B, C, D, F);
        {A,B,C,D} = 4'b0000;
        repeat (15) #10 {A,B,C,D} = {A,B,C,D} + 1;
        #10 $finish;
    end
endmodule
```

Gate-level circuit

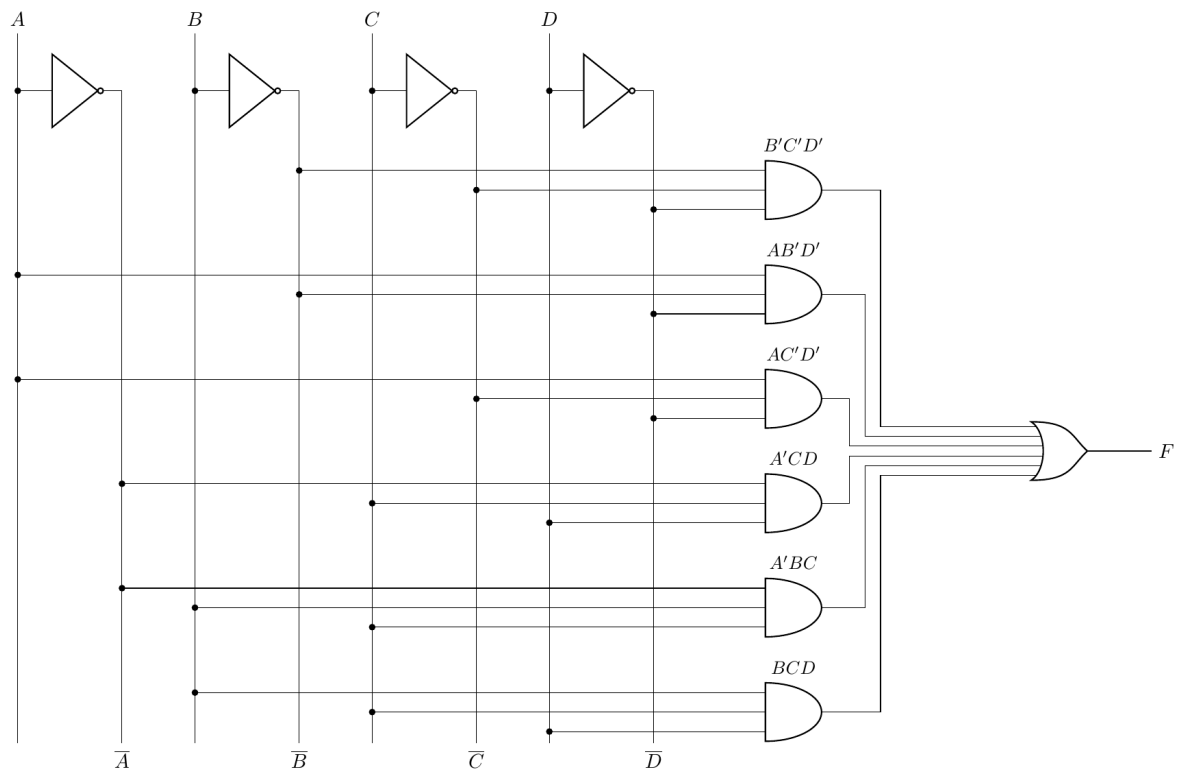


Fig. 2: Four inverters, six 3-input AND gates and one 6-input OR gate (dots mark connections).

Simulation timing diagram

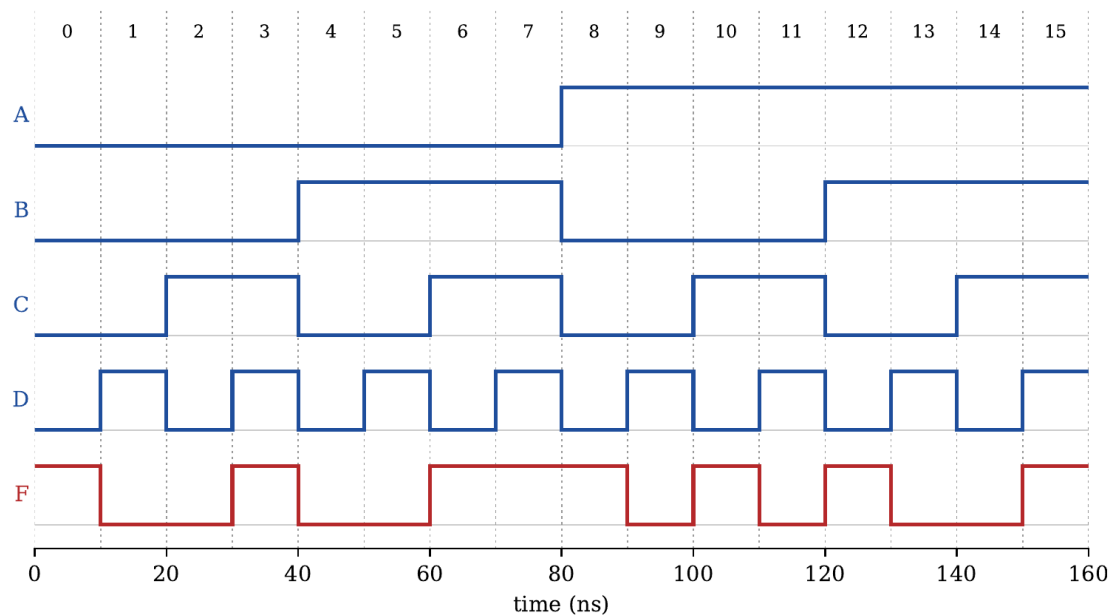


Fig. 3: ABCD counts 0000 $\rightarrow$ 1111 in 10 ns steps (numbers on top are minterm indices). $F = 1$ exactly at m0, m3, m6, m7, m8, m10, m12, m15.

Q 250 B

Implement a 4 to 1 Mux in Verilog HDL using Structural modelling. Draw the gate-level circuit also.

A 4-to-1 multiplexer selects one of four data inputs (D0, D1, D2, D3) and passes it to the single output Y. The choice is made by two select lines A and B.

Boolean expression

$$Y = D0 \cdot A' \cdot B' + D1 \cdot A' \cdot B + D2 \cdot A \cdot B' + D3 \cdot A \cdot B$$

Truth (selection) table

A	B	Selected input	Y
0	0	D0	D0
0	1	D1	D1
1	0	D2	D2
1	1	D3	D3

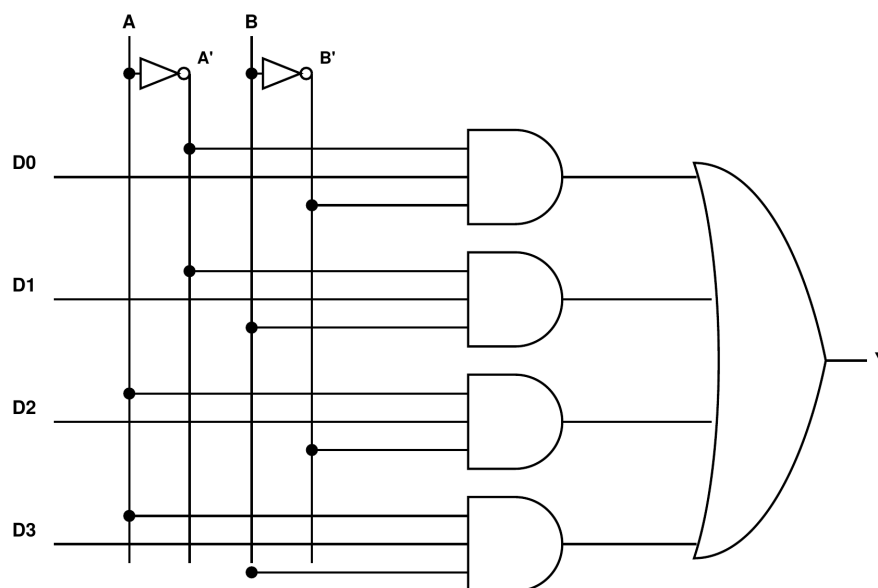
Gate-level circuit

Fig. 1: 4-to-1 MUX using 2 NOT gates, four 3-input AND gates and one 4-input OR gate

Verilog HDL - Structural modelling

```
module mux4to1 (  
    input D0, D1, D2, D3,  
    input A, B,  
    output Y  
);  
    wire A_bar, B_bar;  
    wire W0, W1, W2, W3;  
  
    not (A_bar, A);  
    not (B_bar, B);  
    and (W0, D0, A_bar, B_bar);  
    and (W1, D1, A_bar, B);  
    and (W2, D2, A, B_bar);  
    and (W3, D3, A, B);  
    or (Y, W0, W1, W2, W3);  
endmodule
```

In structural modelling the circuit is described by instantiating the built-in gate primitives (not, and, or) and joining them with wires, exactly as in the gate-level diagram.

Q 251 B

Implement a 1 to 4 Demux in Verilog HDL using Structural modelling. Draw the gate-level circuit also.

A 1-to-4 demultiplexer routes the single data input D to one of four outputs (Y0 to Y3) according to the select lines S1 and S0. Only the selected output receives D; the others stay at 0.

Boolean expressions

$$Y0 = D \cdot S1' \cdot S0'$$

$$Y1 = D \cdot S1' \cdot S0$$

$$Y2 = D \cdot S1 \cdot S0'$$

$$Y3 = D \cdot S1 \cdot S0$$

Truth (selection) table

S1	S0	Y0	Y1	Y2	Y3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

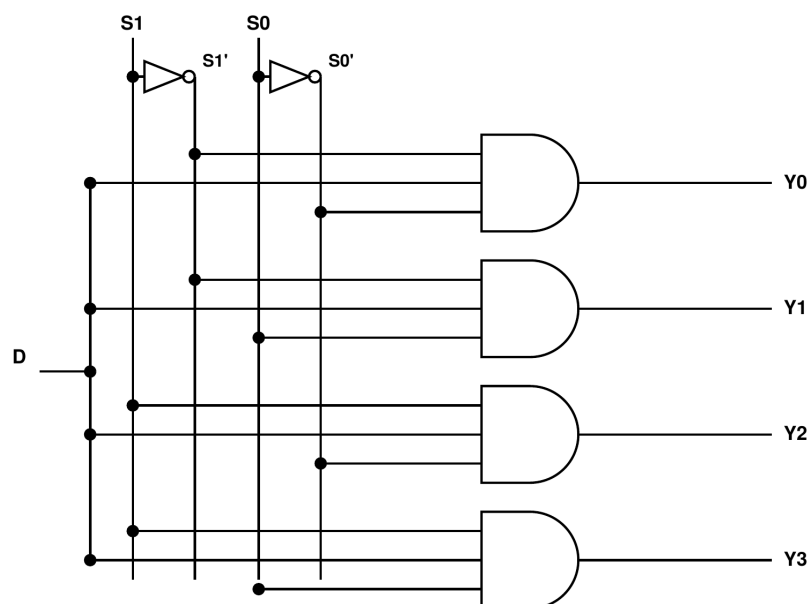
Gate-level circuit

Fig. 1: 1-to-4 DEMUX using 2 NOT gates and four 3-input AND gates

Verilog HDL - Structural modelling

```
module demux1to4 (  
    input D,  
    input S1, S0,  
    output Y0, Y1, Y2, Y3  
);  
    wire S1_bar, S0_bar;  
  
    not (S1_bar, S1);  
    not (S0_bar, S0);  
    and (Y0, D, S1_bar, S0_bar);  
    and (Y1, D, S1_bar, S0);  
    and (Y2, D, S1, S0_bar);  
    and (Y3, D, S1, S0);  
endmodule
```

Q 252 B

Implement a 8 to 1 Mux in Verilog HDL using Structural modelling. Draw the gate-level circuit also.

An 8-to-1 multiplexer has **8 data inputs (D0-D7)**, **3 select inputs (S2, S1, S0)**, and **one output Y**. In structural modelling, NOT, AND and OR gate primitives are instantiated and connected with wires.

Verilog program - Structural modelling

```
module mux8to1(
    input D0, D1, D2, D3, D4, D5, D6, D7,
    input S2, S1, S0,
    output Y
);

    wire nS2, nS1, nS0;
    wire W0, W1, W2, W3, W4, W5, W6, W7;

    not (nS2, S2);
    not (nS1, S1);
    not (nS0, S0);

    and (W0, D0, nS2, nS1, nS0);
    and (W1, D1, nS2, nS1, S0);
    and (W2, D2, nS2, S1, nS0);
    and (W3, D3, nS2, S1, S0);
    and (W4, D4, S2, nS1, nS0);
    and (W5, D5, S2, nS1, S0);
    and (W6, D6, S2, S1, nS0);
    and (W7, D7, S2, S1, S0);

    or (Y, W0, W1, W2, W3, W4, W5, W6, W7);

endmodule
```

Selection table

S2	S1	S0	Selected input	Y
0	0	0	D0	D0
0	0	1	D1	D1
0	1	0	D2	D2
0	1	1	D3	D3
1	0	0	D4	D4
1	0	1	D5	D5
1	1	0	D6	D6
1	1	1	D7	D7

Gate-level circuit

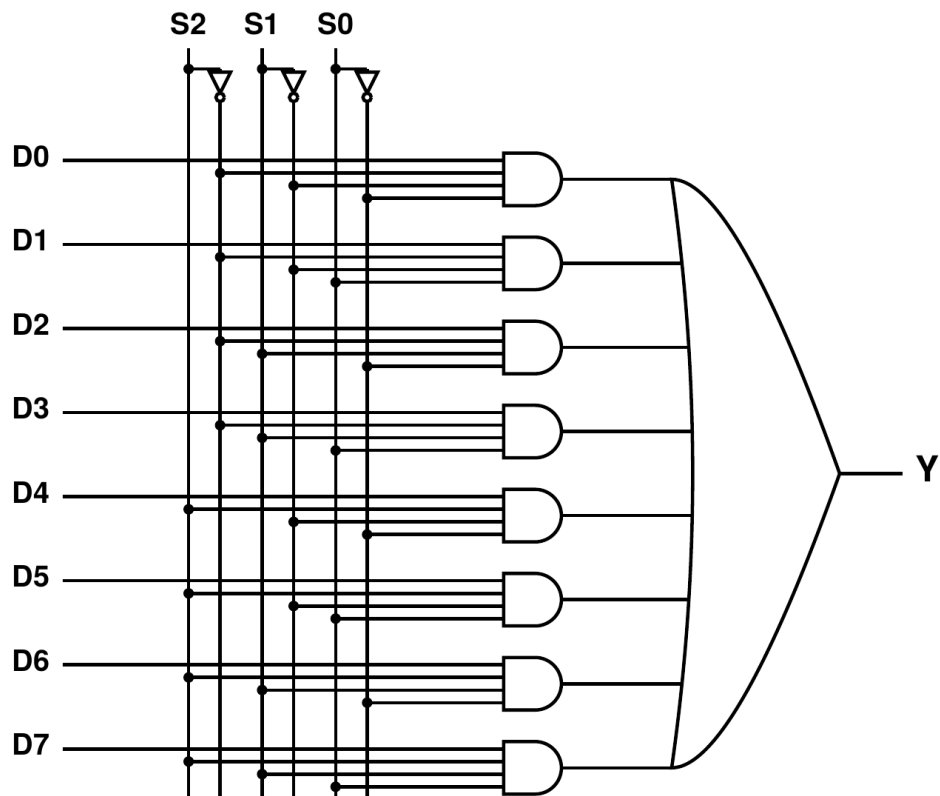


Fig. 1: 8-to-1 MUX (3 NOT gates, eight 4-input AND gates, one 8-input OR gate)

Operation: The select lines S2, S1, S0 enable exactly one AND gate, so only the selected input D0–D7 passes to the OR gate and appears at Y.

Q 253 B

Implement a 1 to 8 Demux in Verilog HDL using Structural modelling. Draw the gate-level circuit also.

A 1-to-8 demultiplexer has **one data input D**, **three select inputs S2, S1, S0**, and **eight outputs Y0-Y7**. The selected output receives D, while the other outputs remain 0.

Verilog program - Structural modelling

```
module demux1to8(
    input D,
    input S2, S1, S0,
    output Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7
);

    wire nS2, nS1, nS0;

    not (nS2, S2);
    not (nS1, S1);
    not (nS0, S0);

    and (Y0, D, nS2, nS1, nS0);
    and (Y1, D, nS2, nS1, S0);
    and (Y2, D, nS2, S1, nS0);
    and (Y3, D, nS2, S1, S0);
    and (Y4, D, S2, nS1, nS0);
    and (Y5, D, S2, nS1, S0);
    and (Y6, D, S2, S1, nS0);
    and (Y7, D, S2, S1, S0);

endmodule
```

Selection table (when D = 1)

S2	S1	S0	Active output
0	0	0	Y0 = 1
0	0	1	Y1 = 1
0	1	0	Y2 = 1
0	1	1	Y3 = 1
1	0	0	Y4 = 1
1	0	1	Y5 = 1
1	1	0	Y6 = 1
1	1	1	Y7 = 1

Gate-level circuit

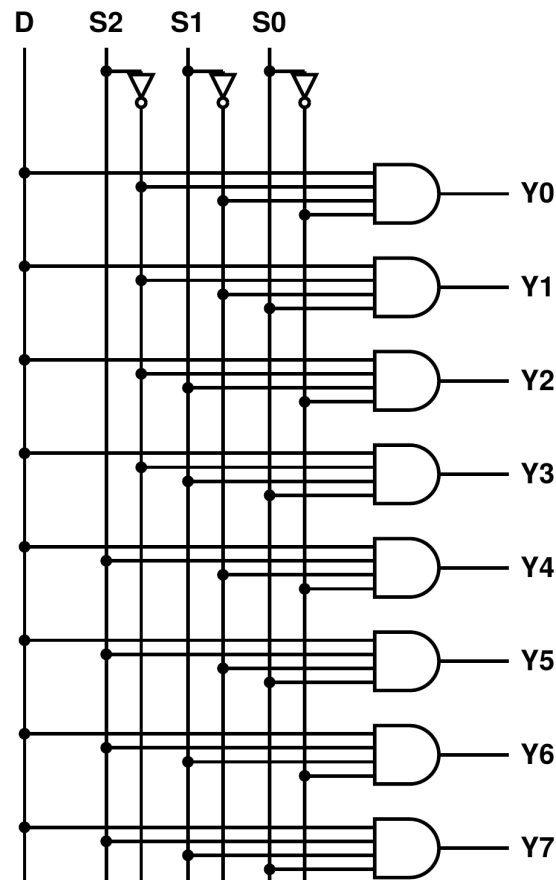


Fig. 1: 1-to-8 DEMUX (3 NOT gates and eight 4-input AND gates)

Operation: The select inputs determine which output receives the input D. For each select combination, only the corresponding output is enabled.

Q 254 B

Implement a 3 to 8 Decoder in Verilog HDL using Structural modelling. Draw the gate-level circuit also.

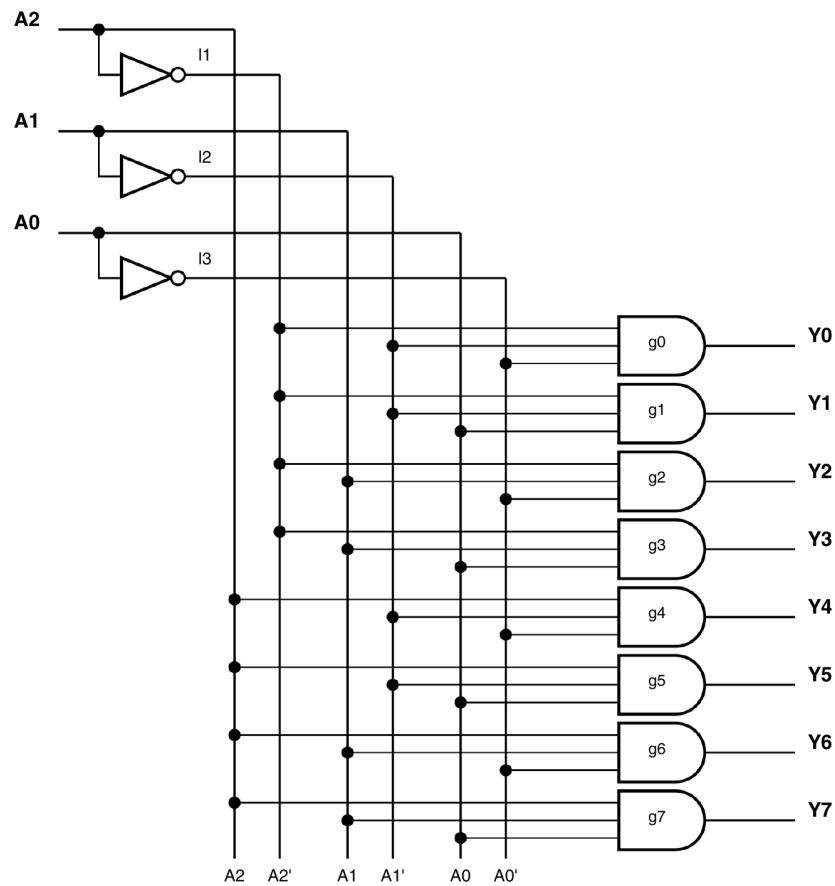
Gate-level circuit

Fig. 1: 3 to 8 decoder (3 NOT gates and 8 three-input AND gates)

Truth table

A2	A1	A0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

Equations

$$Y0 = A2'A1'A0'$$
$$Y1 = A2'A1'A0$$
$$Y2 = A2'A1A0'$$
$$Y3 = A2'A1A0$$
$$Y4 = A2A1'A0'$$
$$Y5 = A2A1'A0$$
$$Y6 = A2A1A0'$$
$$Y7 = A2A1A0$$

Program

```
module Decoder3to8 (input A2, A1, A0,
                    output Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7);
    wire l1, l2, l3;
    not (l1, A2);
    not (l2, A1);
    not (l3, A0);
    and (Y0, l1, l2, l3);
    and (Y1, l1, l2, A0);
    and (Y2, l1, A1, l3);
    and (Y3, l1, A1, A0);
    and (Y4, A2, l2, l3);
    and (Y5, A2, l2, A0);
    and (Y6, A2, A1, l3);
    and (Y7, A2, A1, A0);
endmodule
```

Q 255 B

Write the Verilog HDL program for a 4-bit binary adder built using 4 Full Adders, using module instantiation.

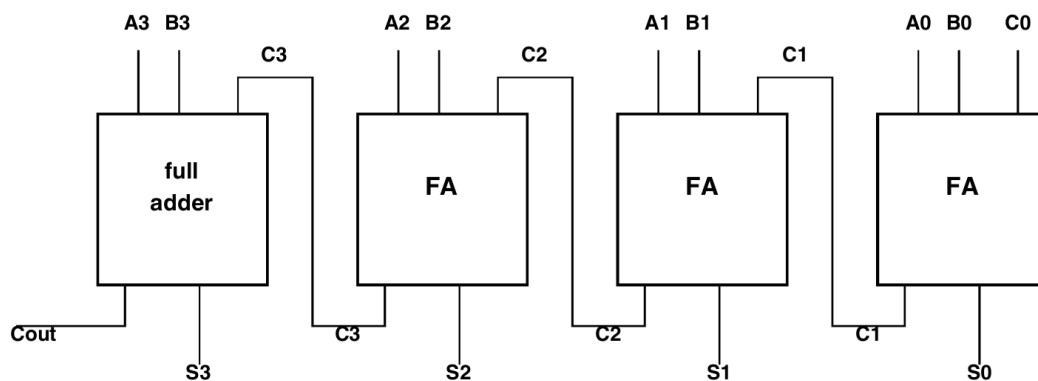
Diagram

Fig. 1: 4-bit binary adder using four full adders

Truth table (Full Adder)

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Equations

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + B\text{Cin} + A\text{Cin}$$

Program

```

module FA (output S, Cout, input A, B, Cin);
    assign S = A ^ B ^ Cin;
    assign Cout = (A & B) | (B & Cin) | (A & Cin);
endmodule

module RCA (input A0, B0, A1, B1, A2, B2, A3, B3, C0,
            output S0, S1, S2, S3, Cout);
    wire C1, C2, C3;
    FA fa0 (S0, C1, A0, B0, C0);
    FA fa1 (S1, C2, A1, B1, C1);
    FA fa2 (S2, C3, A2, B2, C2);
    FA fa3 (S3, Cout, A3, B3, C3);
endmodule

```

Q 256 B**Using the behavioral modeling style in Verilog HDL, write a program to implement a 4-bit binary adder / subtractor.**

The circuit is a 4-bit parallel adder/subtractor controlled by **addnsub**. When **addnsub** = 1 it adds ($a + b$). When **addnsub** = 0 it subtracts using 2's complement: $a - b = a + (\sim b) + 1$. An **always @(*)** block describes the behaviour. **carry_borrow** is the carry-out for addition and the borrow flag for subtraction (1 when $a < b$).

```

module adder_subtractor_4bit (
    input  [3:0] a,
    input  [3:0] b,
    input      addnsub,      // 1 = add, 0 = subtract
    output reg [3:0] result,
    output reg      carry_borrow
);
    reg cout;

    always @(*) begin
        if (addnsub == 1'b1) begin
            // Addition: a + b
            {carry_borrow, result} = {1'b0, a} + {1'b0, b};
        end
        else begin
            // Subtraction using 2's complement: a + (~b) + 1
            {cout, result} = {1'b0, a} + {1'b0, ~b} + 5'd1;
            carry_borrow = ~cout;    // cout = 1 means no borrow
        end
    end
end
endmodule

```

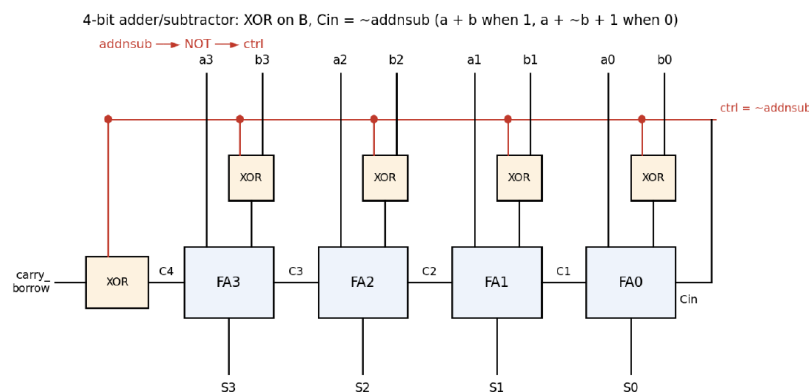
Gate-level realisation

Fig. 1: Four full adders with XOR gates on B; ctrl = $\sim$ addnsub also drives Cin.

Sample results

a	b	addnsub	result	carry_borrow	Meaning
0101	0011	1	1000	0	$5 + 3 = 8$
1001	0111	1	0000	1	$9 + 7 = 16$ (carry out)
0101	0011	0	0010	0	$5 - 3 = 2$ (no borrow)
0011	0101	0	1110	1	$3 - 5 = -2$ (borrow, 2's comp.)

Q 257 B**Implement an 8 to 1 Multiplexer in Verilog HDL using Behavioral model. Use Case statement.**

An 8-to-1 multiplexer passes one of eight data inputs to the output according to a 3-bit select vector **sel**. In behavioural style this is a **case** statement inside an **always @(*)** block.

```

module mux_8to1 (
    input [7:0] in,      // 8 data inputs
    input [2:0] sel,     // 3-bit select
    output reg out       // 1-bit output
);
    always @(*) begin
        case (sel)
            3'b000: out = in[0];
            3'b001: out = in[1];
            3'b010: out = in[2];
            3'b011: out = in[3];
            3'b100: out = in[4];
            3'b101: out = in[5];
            3'b110: out = in[6];
            3'b111: out = in[7];
            default: out = 1'b0; // covers x/z on sel
        endcase
    end
endmodule

```

8-to-1 multiplexer (out = in[sel])

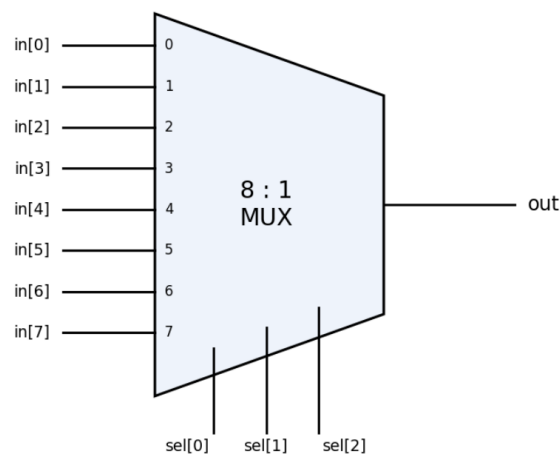


Fig. 1: 8-to-1 multiplexer block diagram

sel[2]	sel[1]	sel[0]	out
0	0	0	in[0]
0	0	1	in[1]
0	1	0	in[2]
0	1	1	in[3]
1	0	0	in[4]
1	0	1	in[5]
1	1	0	in[6]
1	1	1	in[7]

Q 258 B**Implement a JK flip-flop in Verilog HDL using Behavioural model. Use Case statement.**

A JK flip-flop is an edge-triggered memory element with inputs J, K and clock, and outputs Q and Q'. It removes the invalid state of the SR flip-flop: when J = K = 1 the output toggles.

- The behavioural model describes *what* the circuit does, not its gates.
- {j,k} joins the inputs into a 2-bit value used as the case selector.
- Each case item implements one row of the truth table; non-blocking assignments (<=) are used. The default item covers any other value (such as x or z).
- rst is an asynchronous, active-high reset: when rst = 1 the output q is forced to 0 immediately, regardless of clk, j and k.

Function table

J	K	Q(next)	Operation
0	0	Q	Hold (no change)
0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle

Verilog code

```

module jk_ff_case (input j, input k, input clk, input rst,
                  output reg q, output qbar);
    assign qbar = ~q;

    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else begin
            case ({j, k})
                2'b00: q <= q;      // hold
                2'b01: q <= 1'b0;    // reset
                2'b10: q <= 1'b1;    // set
                2'b11: q <= ~q;     // toggle
                default: q <= q;     // safe default
            endcase
        end
    end
endmodule

```

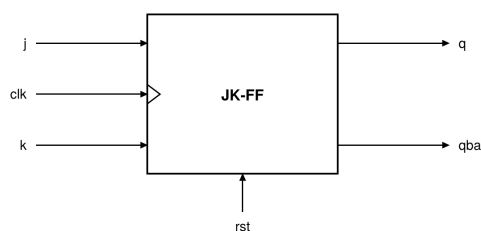
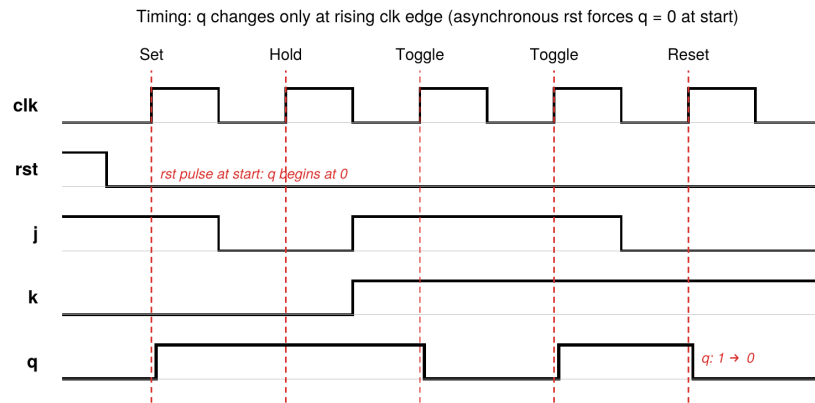


Fig. 1: Block diagram of the JK flip-flop



Timing diagram: rst pulse, then Set, Hold, Toggle, Toggle and Reset on successive rising edges

Fig. 2: Timing diagram - rst pulse, then Set, Hold, Toggle, Toggle and Reset on successive rising edges

Q 259 B**Implement a JK flip-flop in Verilog HDL using Behavioural model. Use if-else statements to describe the functionality.**

The function table and block diagram are the same as Q 258 B; only the coding style changes. The functionality is described with nested if-else statements.

- The conditions are checked from top to bottom; the first true condition executes and the rest are skipped.
- The final else covers the case $J = K = 1$ (toggle).
- The asynchronous reset has the highest priority.

Function table

J	K	Q(next)	Operation
0	0	Q	Hold (no change)
0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle

Verilog code

```

module jk_ff_ifelse (input j, input k, input clk, input rst,
                    output reg q, output qbar);
    assign qbar = ~q;

    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else if (j == 0 && k == 0)
            q <= q;          // hold
        else if (j == 0 && k == 1)
            q <= 1'b0;       // reset
        else if (j == 1 && k == 0)
            q <= 1'b1;       // set
        else
            q <= ~q;         // toggle (J = K = 1)
    end
endmodule

```

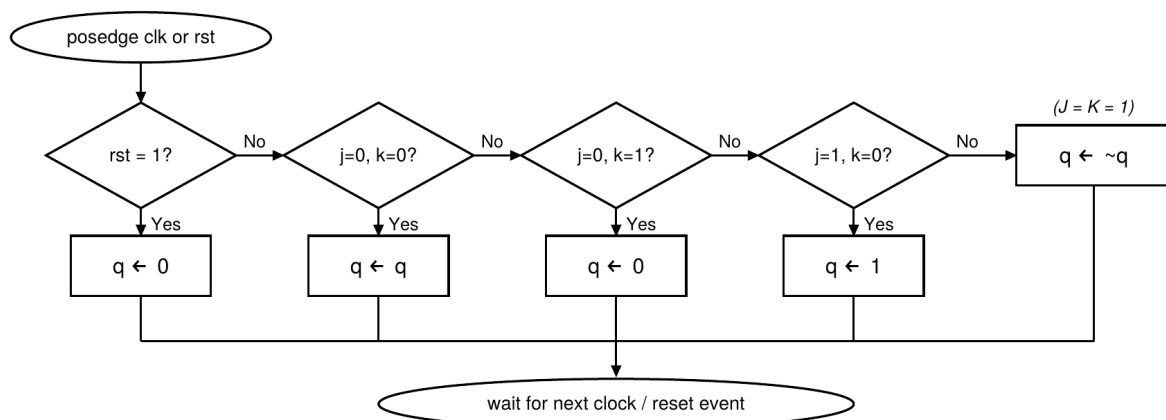


Fig. 1: Flow chart of the if-else description of the JK flip-flop

Q 260 B**Write Verilog code for a 4-bit up counter. Use Behavioural model.****Principle**

A 4-bit up counter is a synchronous sequential circuit. On every **rising edge of the clock** its 4-bit output is incremented by one, giving $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 15 \rightarrow 0 \rightarrow 1 \dots$. It counts from 0 to 15 and then rolls over to 0. An active-high asynchronous **reset** forces the counter to 0000.

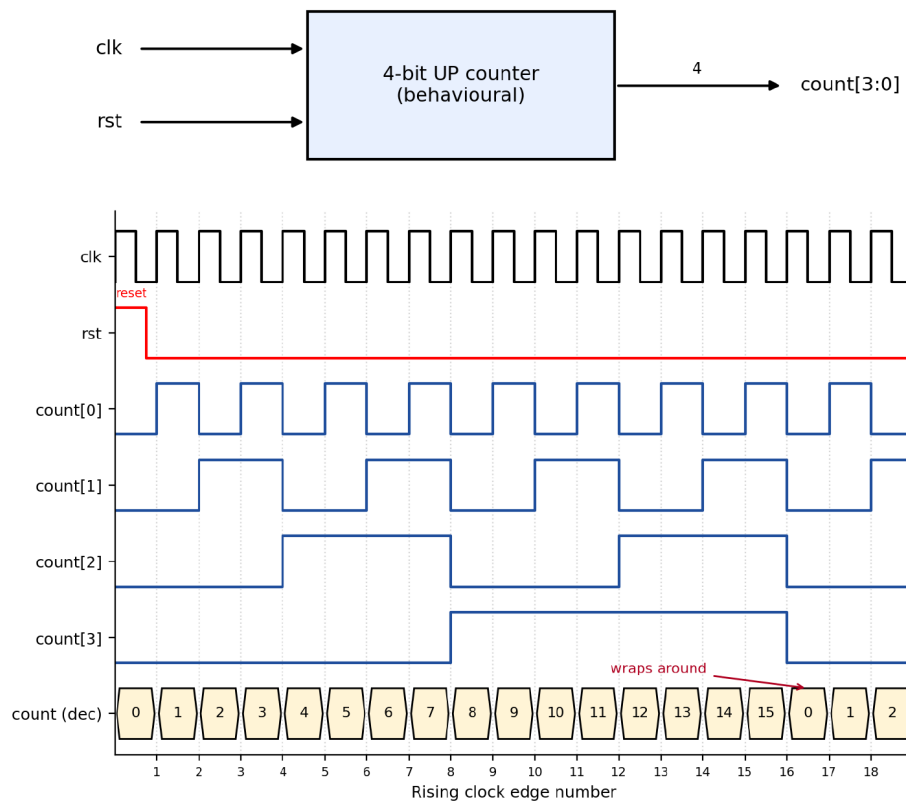


Fig. 1: Block diagram and timing diagram of the 4-bit up counter (reset applied first, then free counting)

Verilog code (behavioural model)

```

module up_counter (
    input    clk,          // clock
    input    rst,          // asynchronous active-high reset
    output reg [3:0] count // 4-bit counter output
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 4'b0000; // reset value (0)
        else
            count <= count + 1; // increment on every clock edge
    end
endmodule

```

Test bench

```

module tb_up_counter;
    reg clk, rst;
    wire [3:0] count;

    up_counter uut (.clk(clk), .rst(rst), .count(count));

    initial clk = 0;
    always #5 clk = ~clk;          // 10 time-unit clock period

    initial begin
        rst = 1; #12;              // apply reset
        rst = 0;                   // release reset
        #170;                      // run for 17 clock cycles
        $finish;
    end

    initial $monitor("t=%0t rst=%b count=%d (%b)", $time, rst, count, count);
endmodule

```

Explanation

1. **output reg [3:0] count** is a 4-bit register because it is assigned inside a procedural block.
2. The sensitivity list **posedge clk or posedge rst** makes the block run on the rising clock edge and immediately when reset goes high (asynchronous reset).
3. When $\text{rst} = 1$, count is set to 0000. Otherwise $\text{count} \leq \text{count} + 1$ executes at each rising edge.
4. The non-blocking assignment $\leq$ is used because the logic is sequential (edge-triggered).
5. A 4-bit register has $2^4 = 16$ states, so after 1111 (15) the counter overflows and wraps to 0000 (0) automatically; no extra code is needed.

Count sequence

Clock edge	0	1	2	3	4	...	14	15	16
count (dec)	0	1	2	3	4	...	14	15	0
count (bin)	0000	0001	0010	0011	0100	...	1110	1111	0000

Clock edge 0 = value after reset.

Q 261 B**Write Verilog code for a 4-bit down counter. Use Behavioural model.****Principle**

A 4-bit down counter is a synchronous sequential circuit. On every **rising edge of the clock** its 4-bit output is decremented by one, giving $15 \rightarrow 14 \rightarrow 13 \rightarrow \dots \rightarrow 0 \rightarrow 15 \rightarrow 14 \dots$. It counts from 15 down to 0 and then rolls over to 15. An active-high asynchronous **reset** forces the counter to 1111.

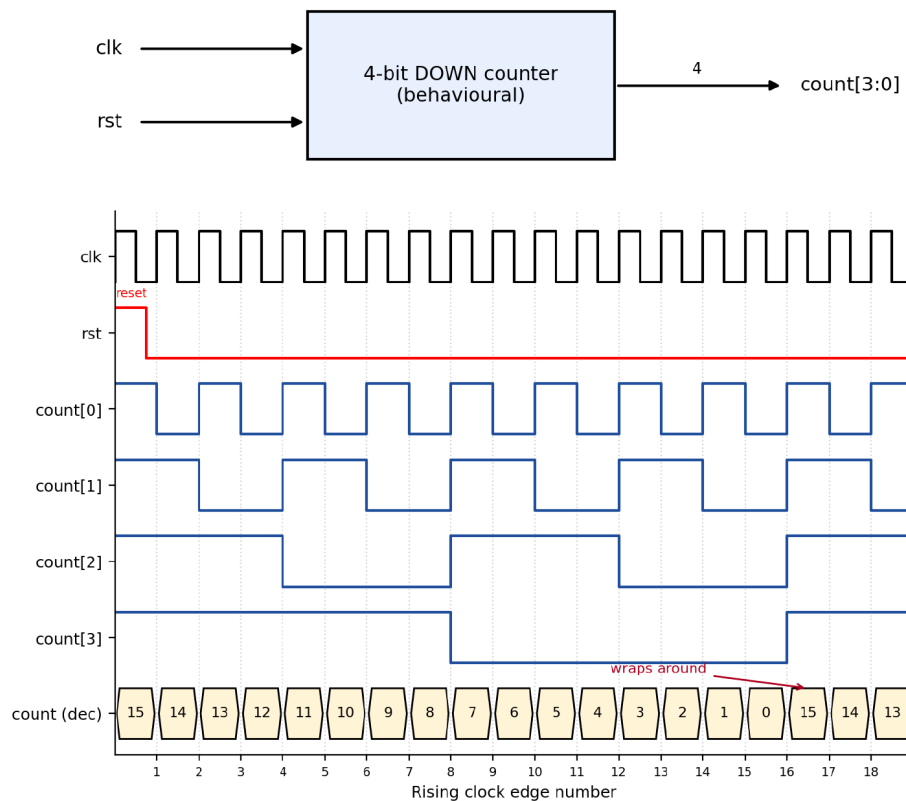


Fig. 1: Block diagram and timing diagram of the 4-bit down counter (reset applied first, then free counting)

Verilog code (behavioural model)

```

module down_counter (
    input    clk,           // clock
    input    rst,           // asynchronous active-high reset
    output reg [3:0] count  // 4-bit counter output
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 4'b1111;      // reset value (15)
        else
            count <= count - 1;      // decrement on every clock edge
    end
endmodule

```

Test bench

```

module tb_down_counter;
    reg clk, rst;
    wire [3:0] count;

    down_counter uut (.clk(clk), .rst(rst), .count(count));

    initial clk = 0;
    always #5 clk = ~clk;          // 10 time-unit clock period

    initial begin
        rst = 1; #12;              // apply reset
        rst = 0;                  // release reset
        #170;                      // run for 17 clock cycles
        $finish;
    end

    initial $monitor("t=%0t rst=%b count=%d (%b)", $time, rst, count, count);
endmodule

```

Explanation

1. **output reg [3:0] count** is a 4-bit register because it is assigned inside a procedural block.
2. The sensitivity list **posedge clk or posedge rst** makes the block run on the rising clock edge and immediately when reset goes high (asynchronous reset).
3. When $\text{rst} = 1$, count is set to 1111. Otherwise $\text{count} \leq \text{count} - 1$ executes at each rising edge.
4. The non-blocking assignment $\leq$ is used because the logic is sequential (edge-triggered).
5. A 4-bit register has $2^4 = 16$ states, so after 0000 (0) the counter underflows and wraps to 1111 (15) automatically; no extra code is needed.

Count sequence

Clock edge	0	1	2	3	4	...	14	15	16
count (dec)	15	14	13	12	11	...	1	0	15
count (bin)	1111	1110	1101	1100	1011	...	0001	0000	1111

Clock edge 0 = value after reset.

Q 262 B

Write Verilog code for a decade counter. Use the Behavioural model.

Aim

To write a behavioural Verilog description of a decade (MOD-10) counter and verify it using a test bench.

Theory

A decade counter counts from **0 to 9** (0000 to 1001) and then returns to 0 on the next clock pulse, i.e. it is a MOD-10 counter. Four flip-flops are required ($2^4 = 16 \geq 10$); the six states 1010 to 1111 are never used. In the behavioural model, an always block sensitive to the rising clock edge increments the count, and the count is cleared to 0 when it reaches 9 or when the asynchronous reset rst is applied.

Count sequence

Clock pulse	Q3	Q2	Q1	Q0	Decimal
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	2
3	0	0	1	1	3
4	0	1	0	0	4
5	0	1	0	1	5
6	0	1	1	0	6
7	0	1	1	1	7
8	1	0	0	0	8
9	1	0	0	1	9
10	0	0	0	0	0 (recycles)

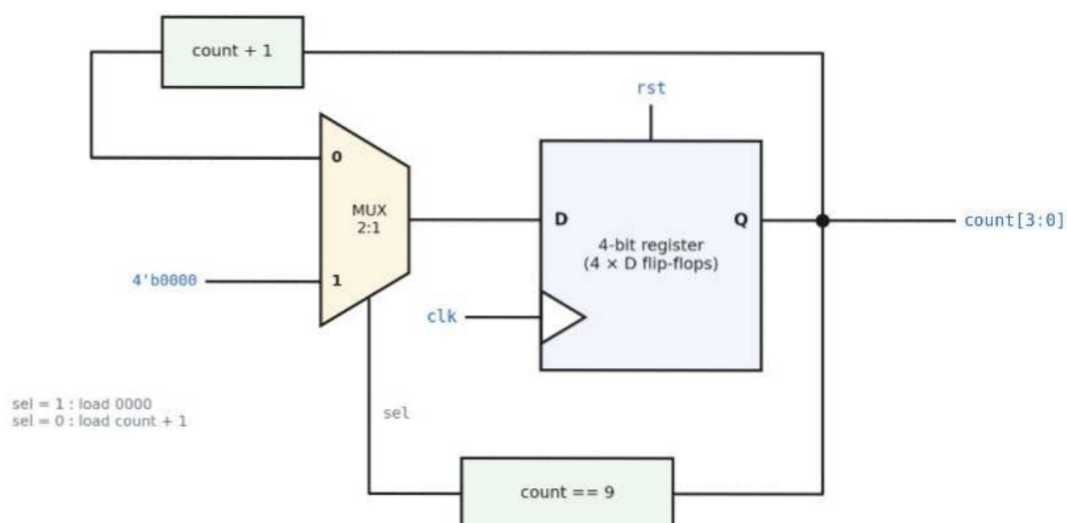
Circuit diagram

Fig. 1: Block diagram of the decade counter (all data paths are 4 bits wide)

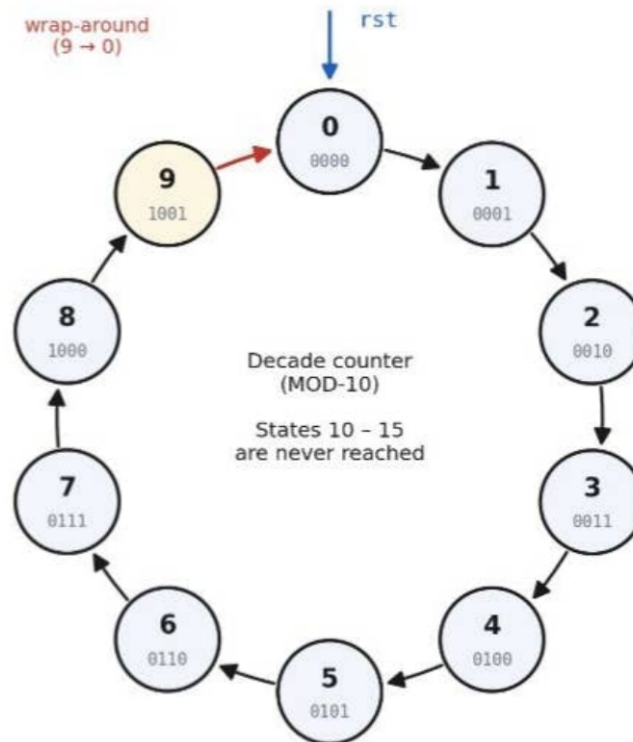


Fig. 2: State diagram - the counter cycles $0 \rightarrow 1 \rightarrow \dots \rightarrow 9 \rightarrow 0$

Verilog code (behavioural model)

```
// Decade (MOD-10 / BCD) counter - Behavioural model
module decade_counter (
    input      clk,           // clock
    input      rst,           // asynchronous active-high reset
    output reg [3:0] count     // 4-bit output : counts 0 to 9
);

always @(posedge clk or posedge rst) begin
    if (rst)
        count <= 4'b0000;           // reset to 0
    else if (count == 4'd9)
        count <= 4'b0000;           // wrap-around after 9
    else
        count <= count + 1'b1;       // normal up-count
end

endmodule
```

Test bench

```
module tb_decade_counter;
    reg        clk, rst;
    wire [3:0] count;

    decade_counter uut (.clk(clk), .rst(rst), .count(count));

    // Clock generation : period = 10 ns
    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $monitor("t=%0t  rst=%b  count=%d (%b)", $time, rst, count, count);
        rst = 1;                // apply reset
        #12 rst = 0;             // release reset, counting starts
        #130 $finish;           // about 13 clock cycles
    end
endmodule
```

Expected output

After reset is released, the count changes on every rising clock edge (t = 15, 25, 35 ... ns):

0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 0 → 1 → 2 → 3 ...

Result: The decade counter was described in behavioural Verilog and the simulation shows a MOD-10 count from 0 to 9 with automatic wrap-around.

Q 272 B

Write Verilog code for a half subtractor using both Dataflow modelling and Structural modelling.

Aim

To implement a half subtractor in Verilog HDL using (a) dataflow modelling and (b) structural modelling, and verify both with a test bench.

Theory

A half subtractor subtracts one 1-bit number from another ($A - B$) and produces two outputs: the **Difference** and the **Borrow**. From the truth table:

- **Difference** = $A \oplus B = A'B + AB'$
- **Borrow** = $A'B$

A	B	Difference (A - B)	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

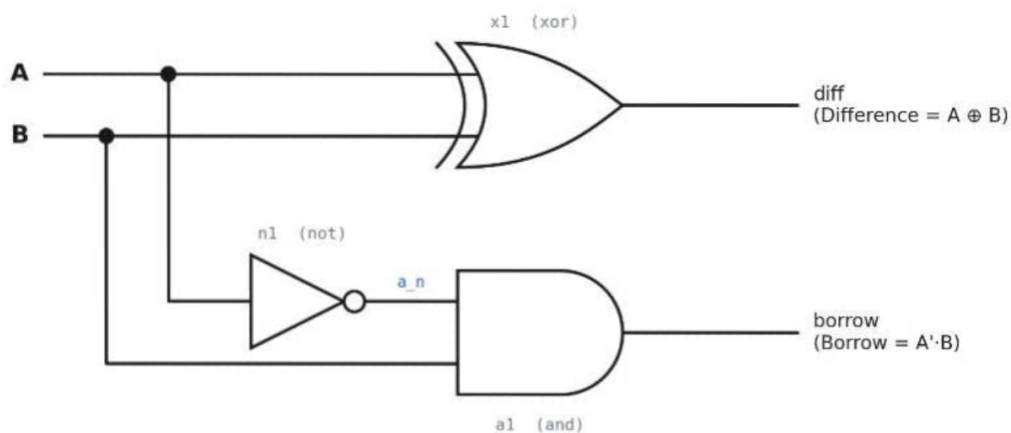
Circuit diagram

Fig. 1: Half subtractor (one XOR gate, one NOT gate and one AND gate). x1, n1, a1 are the instance names used in the structural model.

(a) Verilog code - Dataflow model

```
// Half subtractor - Dataflow modelling (continuous assignments)
module half_sub_dataflow (
    input a, b,
    output diff, borrow
);

    assign diff    = a ^ b;          // Difference = A XOR B
    assign borrow  = (~a) & b;      // Borrow    = A' . B

endmodule
```

(b) Verilog code - Structural model

```
// Half subtractor - Structural modelling (built-in gate primitives)
module half_sub_structural (
    input a, b,
    output diff, borrow
);

    wire a_n;                                // internal wire : complement of A

    // Gate primitive syntax: gate instance_name (output, input1, input2);
    xor x1 (diff, a, b);                     // diff = a ^ b
    not n1 (a_n, a);                         // a_n = ~a
    and a1 (borrow, a_n, b);                 // borrow = a_n & b

endmodule
```

Test bench (for both models)

```
module tb_half_sub;
    reg a, b;
    wire diff_df, borrow_df;                // outputs of dataflow model
    wire diff_st, borrow_st;                // outputs of structural model

    half_sub_dataflow uut1 (.a(a), .b(b), .diff(diff_df), .borrow(borrow_df));
    half_sub_structural uut2 (.a(a), .b(b), .diff(diff_st), .borrow(borrow_st));

    initial begin
        $monitor("A=%b B=%b | DF: diff=%b borrow=%b | ST: diff=%b borrow=%b",
            a, b, diff_df, borrow_df, diff_st, borrow_st);
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;
        $finish;
    end
endmodule
```

Expected output

```
A=0 B=0 | DF: diff=0 borrow=0 | ST: diff=0 borrow=0
A=0 B=1 | DF: diff=1 borrow=1 | ST: diff=1 borrow=1
A=1 B=0 | DF: diff=1 borrow=0 | ST: diff=1 borrow=0
A=1 B=1 | DF: diff=0 borrow=0 | ST: diff=0 borrow=0
```

Result: The half subtractor was implemented using both dataflow and structural modelling; both give identical outputs matching the truth table.

Q 275 B

Write Verilog code for a Full adder using structural modelling. Draw the gate-level circuit also.

A full adder adds three 1-bit inputs A, B and Cin and gives two outputs, Sum and Cout. In structural modelling it is built by instantiating the gate primitives **xor**, **and**, **or** and joining them with wires.

Boolean equations: $\text{Sum} = A \oplus B \oplus \text{Cin}$ and $\text{Cout} = A \cdot B + \text{Cin} \cdot (A \oplus B)$.

Verilog program - Structural modelling

```

module full_adder(input A, B, Cin, output Sum, Cout);
    wire P, G, T;          // internal wires

    xor (P, A, B);          // P = A xor B
    xor (Sum, P, Cin);       // Sum = P xor Cin
    and (G, A, B);          // G = A . B
    and (T, P, Cin);        // T = P . Cin
    or  (Cout, G, T);       // Cout = G + T
endmodule

```

Working: The first XOR gate gives $P = A \oplus B$. The second XOR gate combines P with Cin to give Sum. The two AND gates generate $A \cdot B$ and $P \cdot \text{Cin}$, and the OR gate adds them to produce the carry-out.

Truth table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

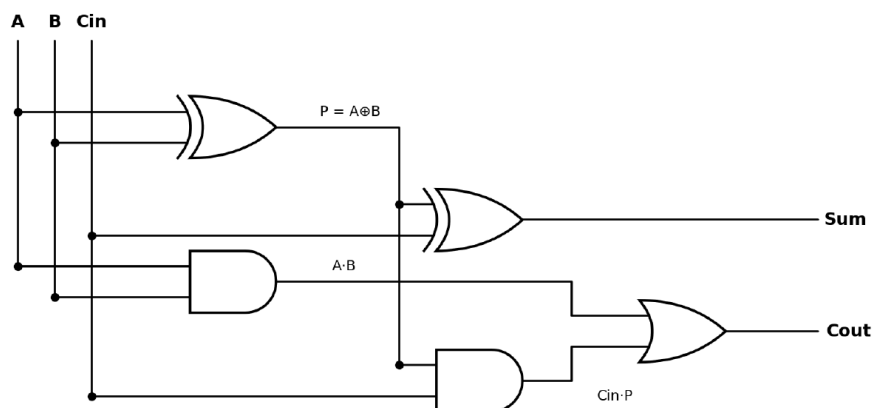
Gate-level circuit

Fig. 1: Full adder (2 XOR, 2 AND, 1 OR)

Operation: Sum is 1 when an odd number of inputs are 1. Cout is 1 when at least two inputs are 1.

Q 276 B

Write Verilog code for a Full adder using structural modelling (two half adders and one OR gate). Draw the gate-level circuit also.

A full adder can also be built structurally from **two half adders and one OR gate**. A half adder (xor + and) is first written as a separate module, and it is instantiated twice inside the full adder module (hierarchical structural modelling). HA1 adds A and B; HA2 adds the partial sum S1 to Cin; the two carries C1 and C2 are ORed to get Cout.

Verilog program - Structural modelling

```
module half_adder(input a, b, output s, c);
    xor (s, a, b);           // sum = a xor b
    and (c, a, b);           // carry = a . b
endmodule

module full_adder_ha(input A, B, Cin, output Sum, Cout);
    wire S1, C1, C2;         // internal wires

    half_adder HA1 (A, B, S1, C1);
    half_adder HA2 (S1, Cin, Sum, C2);
    or (Cout, C1, C2);        // Cout = C1 + C2
endmodule
```

Working: HA1 produces $S1 = A \oplus B$ and $C1 = A \cdot B$. HA2 produces $Sum = S1 \oplus Cin$ and $C2 = S1 \cdot Cin$. Finally $Cout = C1 + C2$, which equals $A \cdot B + Cin \cdot (A \oplus B)$, the same as the full adder equation.

Operation: Each dashed box is one half adder (an XOR gate and an AND gate). Two half adders plus an OR gate give the complete full adder.

Truth table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Gate-level circuit

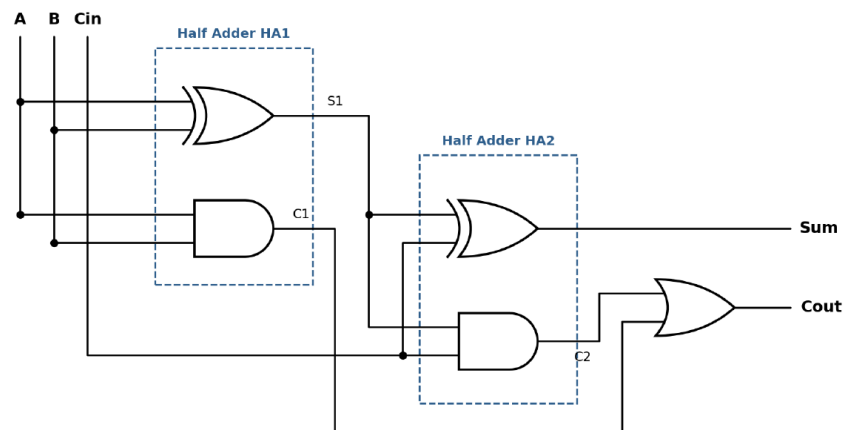


Fig. 1: Full adder from two half adders and an OR gate